

PROJECT ANALYSIS & TECHNICAL BLUEPRINT

FinanceAI Pro

An AI-Powered, Full-Stack Personal Finance Management Platform

Split into Two Independent Deliverables:

PART A — Backend, AI & Machine Learning Architecture

PART B — Frontend Application & User Experience

Prepared by

Md. Kamrujjaman Rahi

Phone / WhatsApp: +880 1882261750

Email: rahiilarham@gmail.com

GitHub: [Kj-RahiL](#)

LinkedIn: [kj-rahiil](#)

July 2026

Table of Contents

Table of Contents	2
1. Project Overview	4
1.1 What is FinanceAI Pro?	4
1.2 Vision & Mission	4
1.3 Target Audience	4
1.4 Core Differentiators (USP)	4
1.5 Suggested Project Names	4
1.6 High-Level System Architecture	5
1.7 Document Structure	5
2. Backend Module Breakdown	7
3. System Architecture (Backend View)	8
4. Database Schema (≈22 Tables)	8
4.1 User & Security (4 tables)	8
4.2 Financial Core (6 tables)	8
4.3 Budget & Recurring (4 tables)	9
4.4 AI & Intelligence (5 tables)	9
4.5 System & ML (3 tables)	9
5. AI / ML Implementation Deep-Dive	9
5.1 Receipt Scanner (Module 5)	9
5.2 AI Chat with Tool Calling (Module 10)	9
5.3 Expense Prediction (Module 9)	10
6. Backend Development Roadmap	10
7. Backend Risks & Mitigation	10
8. Frontend Overview	12
8.1 Recommended Stack	12
9. Screen-by-Screen Breakdown	12
9.1 Authentication Screens	12
9.2 Dashboard (Home)	12
9.3 Accounts & Transactions	13
9.4 AI Receipt Scanner	13
9.5 Budgets	13
9.6 AI Chat (Ask Your Finances)	13
9.7 Reports & Export	13
9.8 Recurring Payments & Loans	13
9.9 Notifications	14
9.10 Settings & Profile	14
10. Frontend Architecture & State Management	14
10.1 Component Structure	14

10.2 State Management Strategy	14
10.3 Real-Time Updates	14
10.4 Handling AI Latency Gracefully.....	14
11. Frontend Development Roadmap.....	15
12. Frontend Risks & Mitigation	15
13. Next Steps	15

1. Project Overview

1.1 What is FinanceAI Pro?

FinanceAI Pro is a next-generation Personal Finance Management (PFM) platform. Traditional finance-tracking apps require users to manually enter every transaction and only ever show them what has already happened. FinanceAI Pro is built around a different idea: automation and intelligence should do the heavy lifting, and the app should also help the user understand what is likely to happen next.

1.2 Vision & Mission

Vision: To empower individuals with AI-driven financial clarity — turning raw transaction data into actionable, predictive, and personalized financial insight.

Mission: Build a secure, scalable, intelligent personal finance platform that automates tracking, forecasts future spending, and answers financial questions in natural language.

1.3 Target Audience

- Tech-savvy professionals who want automated financial control
- Freelancers who need flexible income/expense tracking
- Students managing limited monthly budgets
- Small business owners who want reporting without spreadsheets

1.4 Core Differentiators (USP)

Automation: OCR-based receipt scanning, automatic expense categorization, and recurring-payment tracking — minimal manual data entry.

Intelligence: A natural-language chat interface (RAG) to query your own finances, machine-learning based expense forecasting, and anomaly detection for unusual transactions.

Architecture: A modern microservice architecture — a Node.js service for core application logic and a Python service dedicated to machine learning — built for scalability and clear separation of concerns.

1.5 Suggested Project Names

"FinanceAI Pro" works well as a name, but the following alternatives were also considered:

- WealthMind AI — smart, professional tone
- FiscalFlow AI — emphasizes cash-flow management
- CoinPilot — positions the app as a financial "co-pilot"
- LedgerAI Pro — blends accounting with AI
- SmartSpend AI — direct and easy to understand
- FinSight 360 — implies a complete financial dashboard

- AuraFinance — premium, modern feel
- PredictaWallet — highlights the ML prediction feature

1.6 High-Level System Architecture

The system is composed of three cooperating layers:

- Frontend (Client): a React/Next.js single-page application that users interact with directly.
- Backend (Node.js): the main application server — handles auth, business logic, transactions, budgets, notifications, and orchestrates calls to the AI/ML layer.
- AI/ML Service (Python / FastAPI): a dedicated microservice for machine-learning predictions, running independently so it can be scaled or retrained without touching the main backend.

These three layers communicate over REST APIs (and WebSockets for real-time events), backed by a relational database (PostgreSQL via Prisma), a vector store for semantic search (pgvector), a job queue (BullMQ/Redis) for scheduled and background work, and Cloudinary for file/image storage.

1.7 Document Structure

This document is deliberately split into two self-contained parts so that each half can be handed to a different developer or team without losing context:

Part A — Backend, AI & Machine Learning: database design, API/module breakdown, AI pipelines (OCR, categorization, RAG chat, prediction), job scheduling, and backend-side challenges.

Part B — Frontend: screen-by-screen breakdown, component architecture, state management, data-visualization strategy, and how the frontend consumes the backend/AI APIs.

PART A

Backend, AI & Machine Learning

Database design, APIs, automation pipelines, and predictive intelligence

2. Backend Module Breakdown

The backend is organized into 14 functional modules. Each module owns a clear responsibility and maps to a set of database tables and API endpoints.

#	Module	Key Features	Technical Strategy
1	Auth	Email/Google login, MFA, RBAC, refresh tokens	JWT + httpOnly cookies, bcrypt password hashing, Zod input validation, optional TOTP for MFA
2	Accounts	Cash, Bank, Credit, Mobile, Savings accounts	Relational mapping via Prisma; soft deletes on account closure
3	Transactions	Income, Expense, Transfer, Loan, EMI	Double-entry bookkeeping logic for transfers; idempotency keys for safe API retries
4	Budget	Monthly/Weekly/Category budgets, alerts	BullMQ cron jobs check thresholds daily; alerts via Socket.IO and email
5	AI Receipt Scanner	Extract merchant, date, items, total, VAT, category	Image → Cloudinary → OCR (Google Vision/Tesseract) → LLM with a strict Zod JSON schema output
6	AI Categorization	Auto-tag transactions, e.g. "KFC 550 Tk" → Food	LLM few-shot prompting, or a lightweight fine-tuned classifier in the Python microservice
7	AI Financial Advisor	Spending insights, drop warnings, suggestions	Scheduled BullMQ job aggregates 6 months of data, sends to an LLM for structured insight generation, saves to Reports
8	AI Monthly Report	PDF summaries, charts, recommendations	Puppeteer/PDFKit for PDF generation; data aggregated via Prisma, charted with Recharts/Chart.js
9	Prediction (ML)	Next-month expense & savings forecast	Python (FastAPI) microservice using scikit-learn (Random Forest/Linear Regression) or Prophet
10	AI Chat	"How much did I spend on food?"	RAG + tool calling — LangChain + pgvector semantic search; Node exposes callable tools (e.g. get_spending_by_category)
11	Recurring Payments	Netflix, utilities, auto reminders	BullMQ delayed jobs / cron checks upcoming due dates, sends email/Socket.IO alerts
12	Dashboard	Charts, cash flow, budget vs. actual, predictions	Read-optimized aggregation queries; Redis caching for heavy dashboard metrics

#	Module	Key Features	Technical Strategy
13	Export	CSV, Excel, PDF	json2csv / exceljs for structured export
14	Notification	Email + in-app alerts	Nodemailer + Resend/SendGrid for email, Socket.IO for real-time in-app alerts

3. System Architecture (Backend View)

The backend follows a service-oriented design with a clear split between the main application server and the AI/ML microservice:

- Client apps (web/mobile) → talk only to the Node.js API Gateway/Backend.
- Node.js Backend → owns Auth, Accounts, Transactions, Budgets, Notifications, and orchestrates every other module.
- Python FastAPI ML Service → receives transaction history from the backend, runs forecasting/anomaly models, and returns predictions.
- PostgreSQL (via Prisma) → the system of record for all relational data.
- pgvector → stores embeddings for semantic search used by the RAG-based AI Chat.
- Redis + BullMQ → background jobs, scheduled cron tasks, and caching.
- Cloudinary → stores receipt images and generated PDF reports.
- External LLM Provider (OpenAI/Gemini) → powers OCR post-processing, categorization, chat responses, and financial insight generation.

Communication between Node.js and the Python ML service happens either over HTTP (direct FastAPI calls) or via a message queue (RabbitMQ/Redis Pub-Sub) for asynchronous, non-blocking prediction jobs.

4. Database Schema (≈22 Tables)

Tables are grouped into five logical domains to keep the Prisma schema organized and easy to reason about.

4.1 User & Security (4 tables)

- Users — id, email, password_hash, name, created_at
- Sessions — id, user_id, refresh_token, expires_at
- AuditLogs — id, user_id, action, ip_address, timestamp
- Attachments — id, user_id, url, cloudinary_id, type

4.2 Financial Core (6 tables)

- Accounts — id, user_id, name, type (enum), balance, currency
- Categories — id, name, type (income/expense), icon, is_default

- Transactions — id, account_id, category_id, amount, type, date, description, receipt_id
- Transfers — id, from_account_id, to_account_id, amount, date
- Goals — id, user_id, target_amount, current_amount, deadline
- Tags + TransactionTags (many-to-many) — id, name, color

4.3 Budget & Recurring (4 tables)

- Budgets — id, user_id, category_id, amount, period (weekly/monthly)
- BudgetAlerts — id, budget_id, threshold_percent, is_triggered
- RecurringPayments — id, user_id, name, amount, frequency, next_due_date, category_id
- Loans — id, user_id, type (borrowed/lent), principal, interest_rate, emi_amount

4.4 AI & Intelligence (5 tables)

- Receipts — id, user_id, image_url, merchant, date, total, vat, raw_ocr_text
- ReceiptItems — id, receipt_id, name, price, category
- AIChats — id, user_id, query, response, created_at
- AIHistory — id, user_id, insight_type, generated_data (JSON), created_at
- Reports — id, user_id, month, year, summary_json, pdf_url

4.5 System & ML (3 tables)

- Notifications — id, user_id, type, message, is_read, created_at
- MLPredictions — id, user_id, predicted_expense, confidence_score, month, year
- Anomalies — id, user_id, transaction_id, reason, score

Total: approximately 22 tables — a size that Prisma can manage comfortably with clear relations and minimal duplication.

5. AI / ML Implementation Deep-Dive

5.1 Receipt Scanner (Module 5)

When a user uploads a receipt image, the flow is: image saved to Cloudinary → the URL is sent to a vision-capable AI service (OpenAI Vision API or Google Document AI) → the model's output is forced into a strict Zod schema so the response is always structured JSON, reducing hallucination risk.

5.2 AI Chat with Tool Calling (Module 10)

A LangChain agent is created for conversational queries. When a user asks something like "how much did I spend?", the agent calls a Node.js REST API tool (get_transactions),

retrieves the real data, and then formats a natural-language answer in Bangla or English — grounding the response in actual data rather than the model's own guesses.

5.3 Expense Prediction (Module 9)

At the end of each month, the Node.js backend sends the user's transaction history (as a Pandas DataFrame) to the FastAPI service. FastAPI runs a trained RandomForestRegressor or Prophet model to forecast next month's expenses and stores the result in the MLPredictions table.

6. Backend Development Roadmap

Phase	Timeline	Focus
Phase 1 — Core Foundation	Weeks 1–3	Auth, Accounts, Categories, basic Transaction CRUD, Prisma setup, Dockerization
Phase 2 — AI Automation	Weeks 4–6	AI Receipt Scanner, AI Expense Categorization, Budgets, Recurring Payments, BullMQ jobs
Phase 3 — Intelligence & ML	Weeks 7–9	Python FastAPI microservice setup, Prediction model, AI Financial Advisor insights, RAG-based AI Chat
Phase 4 — Polish & Production	Weeks 10–12	Dashboard charts (API side), PDF export, Notifications, Swagger docs, security audit, deployment

7. Backend Risks & Mitigation

Challenge	Mitigation Strategy
AI hallucination (wrong category or amount)	Always validate LLM output against a Zod schema. Store the raw OCR text alongside the parsed result so users can manually correct mistakes.
Data privacy & security	Sensitive financial data requires database-level encryption, strict RBAC, and anonymizing PII before sending anything to an external AI provider.
ML model cold start	New users don't have enough history for a good forecast. Use simple rule-based estimates for the first ~3 months until 6 months of data accumulate.
Microservice communication	Use HTTP (direct FastAPI calls) for synchronous needs, or a message queue (RabbitMQ/Redis Pub-Sub) for asynchronous, non-blocking prediction jobs.

PART B

Frontend Application

Screens, components, state management, and the user experience layer

8. Frontend Overview

The frontend is the layer users actually touch. Its job is to make every backend and AI capability from Part A feel simple, fast, and trustworthy — showing real-time data, rendering AI-generated insights clearly, and never blocking the user while the AI/ML layer is thinking.

8.1 Recommended Stack

Layer	Recommended Choice	Why
Framework	Next.js (React) + TypeScript	File-based routing, server-side rendering for fast first load, strong typing for a data-heavy finance app
Styling	Tailwind CSS	Fast, consistent design system; easy to keep the dashboard visually clean
State Management	Redux Toolkit (or Zustand for lighter state)	Predictable state for cross-cutting data like auth, accounts, and active budgets
Data Fetching	TanStack Query (React Query)	Caching, background refetch, and loading/error states for REST calls to the Node.js backend
Charts	Recharts / Chart.js	Cash-flow graphs, budget-vs-actual bars, prediction trend lines
Real-time	Socket.IO client	Live budget-alert and notification updates without polling
Forms & Validation	React Hook Form + Zod	Matches the backend's Zod schemas, so validation rules stay consistent client and server side

9. Screen-by-Screen Breakdown

9.1 Authentication Screens

- Login / Register — email+password and Google OAuth options
- MFA Verification — TOTP code entry when MFA is enabled
- Forgot / Reset Password flow

9.2 Dashboard (Home)

The dashboard is the most important screen — it must summarize the user's entire financial picture at a glance.

- Cash-flow summary card (income vs. expense, this month)
- Budget progress bars per category, with color-coded alerts near threshold

- Next-month expense prediction widget (from the ML service)
- Upcoming recurring payments list
- Recent transactions feed
- Quick-add transaction button (and quick receipt-scan shortcut)

9.3 Accounts & Transactions

- Accounts list — cash, bank, credit, mobile, savings, each with running balance
- Transaction list — filterable by account, category, date range, tag
- Add/Edit Transaction modal — supports income, expense, transfer, loan/EMI types
- Transfer flow — clearly shows the double-entry effect (money leaving one account, arriving in another)

9.4 AI Receipt Scanner

A dedicated upload screen: drag-and-drop or camera capture → shows a processing/loading state while OCR + LLM extraction runs → presents the extracted merchant, date, items, total, and VAT for the user to confirm or correct before saving. Editable fields are essential here since AI extraction won't always be perfect.

9.5 Budgets

- Budget creation form — category, amount, period (weekly/monthly)
- Budget detail view — progress bar, remaining amount, days left in period
- Alert threshold settings (e.g. notify at 80% spent)

9.6 AI Chat (Ask Your Finances)

A chat-style interface, similar to a messaging app, where the user types natural-language questions ("How much did I spend on food last month?") and receives grounded answers pulled from their real transaction data — with the option to render a small inline chart when the answer is naturally visual (e.g. a spending breakdown).

9.7 Reports & Export

- Monthly AI report viewer — summary, charts, and recommendations
- Download as PDF / Excel / CSV
- Report history list, organized by month/year

9.8 Recurring Payments & Loans

- Recurring payments list with next due date and quick "mark as paid"
- Loan tracker — borrowed vs. lent, EMI schedule, remaining principal

9.9 Notifications

- In-app notification bell with unread badge (real-time via Socket.IO)
- Notification center — budget exceeded, upcoming bill, anomaly detected

9.10 Settings & Profile

- Profile & security (password, MFA toggle, sessions)
- Category management (custom categories, icons)
- Notification preferences (email vs. in-app)

10. Frontend Architecture & State Management

10.1 Component Structure

A feature-folder structure keeps the codebase scalable as modules grow:

- `/features/auth`, `/features/accounts`, `/features/transactions`, `/features/budgets`, `/features/ai-chat`, `/features/reports` — each owns its own components, hooks, and API calls
- `/components/ui` — shared, reusable primitives (buttons, modals, cards, inputs) styled with Tailwind
- `/components/charts` — wrapped Recharts components with consistent theming across the app

10.2 State Management Strategy

- Global state (Redux/Zustand): authenticated user, active account, theme, notification count
- Server state (React Query): transactions, budgets, predictions, reports — anything fetched from the backend, with automatic caching and refetch
- Local state (useState/useReducer): form inputs, modal open/close, UI-only toggles

10.3 Real-Time Updates

A Socket.IO client connection stays open while the user is logged in, listening for budget-alert and notification events pushed from the backend, and updates the relevant React Query cache entries directly instead of forcing a full page refetch.

10.4 Handling AI Latency Gracefully

AI-backed features (receipt scanning, chat, predictions) can take a few seconds to respond. The frontend should always show a clear loading/thinking state, allow the user to keep using other parts of the app while waiting, and gracefully handle timeouts with a retry option rather than a silent failure.

11. Frontend Development Roadmap

Phase	Timeline	Focus
Phase 1 — Core UI Foundation	Weeks 1–3	Design system in Tailwind, Auth screens, Accounts & Transactions CRUD UI, routing
Phase 2 — Automation UI	Weeks 4–6	Receipt scanner upload/confirm UI, Budget UI with alert indicators, Recurring Payments UI
Phase 3 — Intelligence UI	Weeks 7–9	AI Chat interface, prediction widgets on dashboard, insight/advisor cards
Phase 4 — Polish & Production	Weeks 10–12	Dashboard charts, PDF/Excel export UI, notification center, responsive/mobile polish, accessibility pass

12. Frontend Risks & Mitigation

Challenge	Mitigation Strategy
AI output shown as if it were certain fact	Always keep extracted/predicted values editable, and visually distinguish AI-generated content (e.g. a small "AI suggested" badge) from user-confirmed data.
Perceived slowness during AI calls (OCR, chat, prediction)	Use skeleton loaders and optimistic UI where safe; never block the whole screen for a single AI call.
Data-heavy dashboard feeling cluttered	Prioritize a small number of high-value widgets above the fold; push detailed breakdowns to secondary screens/tabs.
Keeping validation in sync with the backend	Share Zod schemas (or generate frontend types from the Prisma/OpenAPI schema) so frontend and backend validation never drift apart.

13. Next Steps

This document is the blueprint for FinanceAI Pro. From here, work can proceed independently on either side:

Backend track: generate the Prisma schema.prisma file, scaffold the FastAPI ML service, and build out the AI Receipt Scanner endpoint.

Frontend track: set up the Next.js + Tailwind project skeleton, build the Dashboard and Transactions screens first, then layer in the AI Chat and Receipt Scanner UI once their backend endpoints are ready.

Because Part A and Part B are documented separately with clearly defined API boundaries between them, both tracks can be developed in parallel by different people without blocking each other.